Abaqus 2024 Scripting Reference

High-Quality Sample Scripts by Category

This document contains 26 self-contained Abaqus 2024 Python scripting patterns, retrieved by Consultant and Coding Agent as example patterns for common scripting operations -- NOT an exhaustive API reference, and not a substitute for official Abaqus/SIMULIA documentation.

On correctness: every script below has been validated for Python SYNTAX correctness (parsed without error). They have NOT been executed against a live Abaqus installation, and API-level correctness (correct method names, correct argument names/types for the current Abaqus 2024 release) reflects careful, well-established knowledge of the Abaqus scripting API rather than live verification. Spot-check a sample against a real Abaqus install before treating this set as fully verified.

On named regions: many scripts below operate on a named set, surface, or region (e.g. SET_NAME, SURFACE_NAME). Every such script explicitly checks that the named region already exists before proceeding, and raises a clear error if it does not. NEVER remove this check, and NEVER replace a missing named region with a programmatically guessed geometric selection (coordinate-based, bounding-box, or otherwise) -- if the required region does not exist in the model, that is a genuine gap that needs to be resolved upstream (the region must be defined in the model itself), not something a script should work around silently.

Template used for every script below: Title & Category, Description, Prerequisites/Assumptions, Inputs Required, Key API Objects/Methods, the Script itself, and Notes/Caveats.

Contents

Part / Geometry
- Import STEP Geometry and Create a Part

Materials & Sections
- Isotropic Elastic Material and Whole-Part Solid Section
- Region-Specific Material and Section Assignment
- Shell Section with Thickness

Assembly
- Create a Part Instance

Mesh
- Global Seed and Mesh a Whole Part
- Region-Specific Local Seed and Mesh Refinement

Steps
- Static General Step
- Nonlinear Static Step with Increment Control

Boundary Conditions
- Encastre Boundary Condition
- Displacement / Rotation Boundary Condition
- Symmetry Boundary Condition

Loads
- Concentrated Force on a Node Set
- Pressure Load on a Surface
- Distributed Line Load on Beam Elements
- Gravity / Body Force Load

Interactions & Connections
- Tie Constraint Between Two Surfaces
- Surface-to-Surface Contact with Friction
- Kinematic Coupling Constraint

Job Submission
- Submit a Job Directly from an Input File

Post-Processing
- Peak von Mises Stress Extraction
- Reaction Force Extraction at a Node Set
- Nodal Displacement Time-History
- Write Extracted Stress Results to CSV

File Utility
- List All Defined Sets and Surfaces in an Existing Model
- Write an Updated Input File (.inp) from the Current In-Memory Model

1. Import STEP Geometry and Create a Part

Category: Part / Geometry

Description

Imports raw CAD geometry from a .step/.stp file and creates a corresponding Abaqus part. This is the starting point for any task that begins from uploaded CAD geometry.

Prerequisites / Assumptions

A valid .step or .stp file must exist at STEP_FILE_PATH.

Inputs Required

- **MODEL_NAME** (str): Name for the model database entry, e.g. "Model-1"
- **STEP_FILE_PATH** (str): Absolute path to the .step/.stp file
- **PART_NAME** (str): Name to assign to the imported part

Key API Objects / Methods Used

mdb.openStep(), model.PartFromGeometryFile()

Script

```
from abaqus import mdb
import abaqusConstants as C

# === USER INPUTS ===
MODEL_NAME = "Model-1"
STEP_FILE_PATH = "/path/to/geometry.stp"
PART_NAME = "ImportedPart"
# ====================

model = mdb.models[MODEL_NAME]

step_import = mdb.openStep(STEP_FILE_PATH, scaleFromFile=C.OFF)
model.PartFromGeometryFile(
    name=PART_NAME,
    geometryFile=step_import,
    combine=False,
    dimensionality=C.THREE_D,
    type=C.DEFORMABLE_BODY,
)

print("Imported STEP geometry from '{}' and created part '{}'".format(STEP_FILE_PATH, PART_
NAME))
```

Notes / Caveats

PERSISTENCE: this script assumes MODEL_NAME's default fresh model (auto-created when a new mdb session starts) -- there is nothing to open from an existing .inp yet at this stage. If this is the ONLY operation in this script's dispatch, add a final job.writeInput() call (see the Write an Updated Input File script) to persist the imported part before this process ends -- otherwise, if instancing/meshing continue in the SAME script (the typical case), only write out the .inp ONCE, at the true end of the combined script, not after every individual sub-step.

2. Isotropic Elastic Material and Whole-Part Solid Section

Category: Materials & Sections

Description

Opens an existing model from its .inp file, creates a simple isotropic elastic material and a matching homogeneous solid section, assigns that section to an entire part, and writes the updated model back out to a new .inp. Use this when the whole part is a single material.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file already containing the part PART_NAME with solid (3D) geometry.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **PART_NAME** (str): Name of the part to assign the section to (confirm this against the actual model -- see the List All Defined Sets utility script if uncertain)
- **MATERIAL_NAME** (str): Name for the new material, e.g. "Steel"
- **YOUNGS_MODULUS** (float): Young's modulus, in consistent units (e.g. MPa)
- **POISSONS_RATIO** (float): Poisson's ratio
- **SECTION_NAME** (str): Name for the new solid section
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), model.Material(), model.HomogeneousSolidSection(), part.Set(), part.SectionAssignment(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
PART_NAME = "Part-1"
MATERIAL_NAME = "Steel"
YOUNGS_MODULUS = 200000.0  # MPa
POISSONS_RATIO = 0.3
SECTION_NAME = "Solid-Section-1"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)
```

```
# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
part = model.parts[PART_NAME]

material = model.Material(name=MATERIAL_NAME)
material.Elastic(table=((YOUNGS_MODULUS, POISSONS_RATIO),))

model.HomogeneousSolidSection(name=SECTION_NAME, material=MATERIAL_NAME)

all_cells = part.cells[:]
region = part.Set(cells=all_cells, name="WholePart-Set")
part.SectionAssignment(region=region, sectionName=SECTION_NAME)

# Persist the updated model -- this process's in-memory model does not
# survive once this script finishes; a later, separately-dispatched script
# must be able to pick up from here.
export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Assigned material '{}' via section '{}' to entire part '{}'. Wrote updated model to
'{}.inp'".format(
    MATERIAL_NAME, SECTION_NAME, PART_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

If the part is shell/2D geometry rather than solid, use faces instead of cells when building the Set. The exact PART_NAME after ModelFromInputFile depends on how the original .inp was authored -- confirm it via the List All Defined Sets utility script if it's not already known.

3. Region-Specific Material and Section Assignment

Category: Materials & Sections

Description

Opens an existing model, assigns a material and section to only a specific named region of a part (rather than the whole part), and persists the result -- for tasks like "the hub should be aluminum, the shaft should be steel" where both are modeled as one part.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SET_NAME must already exist as a defined set on the part. This script does NOT invent, guess, or programmatically derive this set -- it checks for its existence and raises an error if not found.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **PART_NAME** (str): Name of the part containing the target region
- **SET_NAME** (str): The exact, pre-existing set name for the target region
- **MATERIAL_NAME** (str): Name for the new material
- **YOUNGS_MODULUS** (float): Young's modulus, in consistent units
- **POISSONS_RATIO** (float): Poisson's ratio
- **SECTION_NAME** (str): Name for the new section
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), part.sets, model.Material(), model.HomogeneousSolidSection(), part.SectionAssignment(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
PART_NAME = "Part-1"
SET_NAME = "Hub-Region"
MATERIAL_NAME = "Aluminum"
YOUNGS_MODULUS = 69000.0  # MPa
POISSONS_RATIO = 0.33
SECTION_NAME = "Hub-Section"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
part = model.parts[PART_NAME]
```

```
# This script assumes SET_NAME already exists as a defined set on the part.
# It does not invent, guess, or programmatically derive this name -- confirm
# the exact set name exists among the part's defined sets before running this.
if SET_NAME not in part.sets.keys():
    raise ValueError(
        "Set '{}' does not exist on part '{}'. Confirm the exact set name "
        "before proceeding.".format(SET_NAME, PART_NAME)
    )

material = model.Material(name=MATERIAL_NAME)
material.Elastic(table=((YOUNGS_MODULUS, POISSONS_RATIO),))

model.HomogeneousSolidSection(name=SECTION_NAME, material=MATERIAL_NAME)

region = part.sets[SET_NAME]
part.SectionAssignment(region=region, sectionName=SECTION_NAME)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Assigned material '{}' via section '{}' to region '{}'. Wrote updated model to '{}.i
np'".format(
    MATERIAL_NAME, SECTION_NAME, SET_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

If the target set is actually defined at the assembly level rather than the part level, reference it via model.rootAssembly.sets instead of part.sets.

4. Shell Section with Thickness

Category: Materials & Sections

Description

Opens an existing model, creates an isotropic elastic material and a homogeneous shell section with a specified thickness, assigns it to an entire shell part, and persists the result.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file already containing the part PART_NAME with shell (2D surface) geometry.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **PART_NAME** (str): Name of the shell part
- **MATERIAL_NAME** (str): Name for the new material
- **YOUNGS_MODULUS** (float): Young's modulus, in consistent units
- **POISSONS_RATIO** (float): Poisson's ratio
- **SECTION_NAME** (str): Name for the new shell section
- **THICKNESS** (float): Shell thickness, in consistent units (e.g. mm)

- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), model.Material(), model.HomogeneousShellSection(), part.Set(), part.SectionAssignment(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
PART_NAME = "Part-1"
MATERIAL_NAME = "Steel"
YOUNGS_MODULUS = 200000.0
POISSONS_RATIO = 0.3
SECTION_NAME = "Shell-Section-1"
THICKNESS = 2.0  # mm
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
part = model.parts[PART_NAME]

material = model.Material(name=MATERIAL_NAME)
material.Elastic(table=((YOUNGS_MODULUS, POISSONS_RATIO),))

model.HomogeneousShellSection(name=SECTION_NAME, material=MATERIAL_NAME, thickness=THICKNES
S)

all_faces = part.faces[:]
region = part.Set(faces=all_faces, name="WholePart-Shell-Set")
part.SectionAssignment(region=region, sectionName=SECTION_NAME)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
```

```
export_job.writeInput(consistencyChecking=False)

print("Created shell section '{}' (thickness {}) and assigned to part '{}'. Wrote updated m
odel to '{}.inp'".format(
    SECTION_NAME, THICKNESS, PART_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

None.

5. Create a Part Instance

Category: Assembly

Description

Creates an instance of a part in the root assembly, as either dependent (mesh comes from the part) or independent (instance has its own mesh).

Prerequisites / Assumptions

The part PART_NAME must already exist in the CURRENT session (typically just created via the STEP import script, in the SAME script) -- an .inp file already contains its own assembly/instance definitions, so this script is not meant to resume from an existing .inp.

Inputs Required

- **MODEL_NAME** (str): Name of the model database entry (matching whatever the geometry-import step used)
- **PART_NAME** (str): Name of the part to instance
- **INSTANCE_NAME** (str): Name for the new instance
- **DEPENDENT** (bool): True for a dependent instance, False for independent

Key API Objects / Methods Used

assembly.Instance()

Script

```
from abaqus import mdb
import abaqusConstants as C

# === USER INPUTS ===
MODEL_NAME = "Model-1"
PART_NAME = "Part-1"
INSTANCE_NAME = "Part-1-1"
DEPENDENT = True
# ====================

model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly
part = model.parts[PART_NAME]

dependent_flag = C.ON if DEPENDENT else C.OFF
assembly.Instance(name=INSTANCE_NAME, part=part, dependent=dependent_flag)

print("Created instance '{}' of part '{}' ({})".format(
    INSTANCE_NAME, PART_NAME, "dependent" if DEPENDENT else "independent"
))
```

Notes / Caveats

PERSISTENCE: same as the STEP import script -- if this is the last operation in its dispatch, add a final job.writeInput() (see the Write an Updated Input File script) before this process ends. If meshing continues in the same script, write out only once, at the true end.

6. Global Seed and Mesh a Whole Part

Category: Mesh

Description

Applies a uniform global seed size to an entire part and generates the mesh. Use this when meshing applies uniformly across the whole part, with no region-specific refinement.

Prerequisites / Assumptions

The part PART_NAME must already exist in the CURRENT session with native CAD geometry (from a STEP import, in the SAME script) -- NOT reconstructed from a plain .inp file. A typical .inp is already a mesh-based solver format and generally has no native geometry for Abaqus's meshing tools to operate on; this script is for the geometry-import stage, before any .inp has been written for this model.

Inputs Required

- **MODEL_NAME** (str): Name of the model database entry (matching whatever the geometry-import step used)
- **PART_NAME** (str): Name of the part to mesh
- **GLOBAL_SEED_SIZE** (float): Approximate global element size, in consistent units

Key API Objects / Methods Used

part.seedPart(), part.generateMesh()

Script

```
from abaqus import mdb

# === USER INPUTS ===
MODEL_NAME = "Model-1"
PART_NAME = "Part-1"
GLOBAL_SEED_SIZE = 5.0
# ====================

model = mdb.models[MODEL_NAME]
part = model.parts[PART_NAME]

part.seedPart(size=GLOBAL_SEED_SIZE, deviationFactor=0.1, minSizeFactor=0.1)
part.generateMesh()

num_elements = len(part.elements)
print("Meshed part '{}' with global seed size {} -- {} elements generated".format(
    PART_NAME, GLOBAL_SEED_SIZE, num_elements
))
```

Notes / Caveats

PERSISTENCE: meshing is typically the natural checkpoint at the end of the geometry-import stage -- once meshed, add a final job.writeInput() (see the Write an Updated Input File script) to persist the meshed model to .inp, so later, separately-dispatched scripts (materials, loads, etc.) can open it from there. If generateMesh() completes without error but len(part.elements) is 0, the geometry likely has an unmeshable topology -- surface this as a genuine failure, not a silent success.

7. Region-Specific Local Seed and Mesh Refinement

Category: Mesh

Description

Applies a finer local seed to a specific named edge set, in addition to a coarser global seed, then meshes the part. Use this for tasks like "refine the mesh near the fillet".

Prerequisites / Assumptions

Same as the global-mesh script: the part must exist in the CURRENT session with native geometry, not reconstructed from a plain .inp. EDGE_SET_NAME must already exist as a defined set of edges on the part.

Inputs Required

- **MODEL_NAME** (str): Name of the model database entry
- **PART_NAME** (str): Name of the part to mesh
- **EDGE_SET_NAME** (str): The exact, pre-existing edge set name for local refinement
- **LOCAL_SEED_SIZE** (float): Local element size at the refined region
- **GLOBAL_SEED_SIZE** (float): Global element size elsewhere on the part

Key API Objects / Methods Used

part.seedPart(), part.seedEdgeBySize(), part.generateMesh()

Script

```
from abaqus import mdb
import abaqusConstants as C

# === USER INPUTS ===
MODEL_NAME = "Model-1"
PART_NAME = "Part-1"
EDGE_SET_NAME = "Fillet-Edges"
LOCAL_SEED_SIZE = 1.0
GLOBAL_SEED_SIZE = 5.0
# ====================

model = mdb.models[MODEL_NAME]
part = model.parts[PART_NAME]

# This script assumes EDGE_SET_NAME already exists as a defined set of edges
# on the part. It does not invent or guess this name -- confirm it exists
# before running this script.
if EDGE_SET_NAME not in part.sets.keys():
    raise ValueError("Set '{}' does not exist on part '{}'.".format(EDGE_SET_NAME, PART_NAM
E))

part.seedPart(size=GLOBAL_SEED_SIZE, deviationFactor=0.1, minSizeFactor=0.1)

target_edges = part.sets[EDGE_SET_NAME].edges
part.seedEdgeBySize(edges=target_edges, size=LOCAL_SEED_SIZE, deviationFactor=0.1, constrai
nt=C.FINER)

part.generateMesh()

num_elements = len(part.elements)
print("Meshed part '{}' with local refinement on '{}' -- {} elements generated".format(
    PART_NAME, EDGE_SET_NAME, num_elements
))
```

Notes / Caveats

PERSISTENCE: same as the global-mesh script -- add a final job.writeInput() once meshing is complete. constraint=C.FINER only allows the local seed to make the mesh finer than the global seed at that edge, not coarser.

8. Static General Step

Category: Steps

Description

Opens an existing model, creates a standard linear static analysis step with specified increment control, and persists the result.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. PREVIOUS_STEP must already exist in that model (typically "Initial", or an earlier named step).

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): Name for the new step
- **PREVIOUS_STEP** (str): Name of the step this one follows
- **INITIAL_INC** (float): Initial increment size
- **MAX_INC** (float): Maximum increment size
- **MIN_INC** (float): Minimum increment size
- **MAX_NUM_INC** (int): Maximum number of increments allowed
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), model.StaticStep(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1"
PREVIOUS_STEP = "Initial"
INITIAL_INC = 0.1
MAX_INC = 1.0
MIN_INC = 1e-05
MAX_NUM_INC = 100
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)
```

```
# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]

model.StaticStep(
    name=STEP_NAME,
    previous=PREVIOUS_STEP,
    initialInc=INITIAL_INC,
    maxInc=MAX_INC,
    minInc=MIN_INC,
    maxNumInc=MAX_NUM_INC,
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Created static general step '{}' after '{}'. Wrote updated model to '{}.inp'".format
(
    STEP_NAME, PREVIOUS_STEP, OUTPUT_JOB_NAME
))
```

Notes / Caveats

None.

9. Nonlinear Static Step with Increment Control

Category: Steps

Description

Opens an existing model and creates a static step with geometric nonlinearity enabled (nlgeom=ON) -- required when large displacements/rotations or nonlinear buckling behavior are expected to matter.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. PREVIOUS_STEP must already exist.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): Name for the new step
- **PREVIOUS_STEP** (str): Name of the step this one follows
- **INITIAL_INC** (float): Initial increment size (typically smaller than a linear step)
- **MAX_INC** (float): Maximum increment size
- **MIN_INC** (float): Minimum increment size (typically smaller than a linear step)

- **MAX_NUM_INC** (int): Maximum number of increments allowed (typically higher than a linear step)
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), model.StaticStep() with nlgeom=ON, job.writeInput()

Script

```
from abaqus import mdb
import os
import abaqusConstants as C

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1-NL"
PREVIOUS_STEP = "Initial"
INITIAL_INC = 0.01
MAX_INC = 0.1
MIN_INC = 1e-06
MAX_NUM_INC = 200
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]

model.StaticStep(
    name=STEP_NAME,
    previous=PREVIOUS_STEP,
    nlgeom=C.ON,
    initialInc=INITIAL_INC,
    maxInc=MAX_INC,
    minInc=MIN_INC,
    maxNumInc=MAX_NUM_INC,
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
```

```
export_job.writeInput(consistencyChecking=False)

print("Created nonlinear (nlgeom=ON) static step '{}' after '{}'. Wrote updated model to '{
}.inp'".format(
    STEP_NAME, PREVIOUS_STEP, OUTPUT_JOB_NAME
))
```

Notes / Caveats

Smaller initial/minimum increments and a higher maxNumInc are typically needed for nonlinear convergence compared to an equivalent linear step -- the defaults shown are conservative starting points, not universally correct for every model.

10. Encastre Boundary Condition

Category: Boundary Conditions

Description

Opens an existing model and fully fixes (all 6 degrees of freedom) a named region -- the standard "fixed support" boundary condition.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SET_NAME must already exist as a defined set at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): Step this BC is created in (often "Initial" for a support fixed throughout)
- **SET_NAME** (str): The exact, pre-existing assembly-level set name for the fixed region
- **BC_NAME** (str): Name for the new boundary condition
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, model.EncastreBC(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Initial"
SET_NAME = "Fixed-Support"
BC_NAME = "Encastre-1"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )
```

```
mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

# This script assumes SET_NAME already exists as a defined set at the
# assembly level. It does not invent or guess this name -- confirm it
# exists before running this script.
if SET_NAME not in assembly.sets.keys():
    raise ValueError("Set '{}' does not exist in the assembly.".format(SET_NAME))

region = assembly.sets[SET_NAME]
model.EncastreBC(name=BC_NAME, createStepName=STEP_NAME, region=region)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Applied encastre BC '{}' to set '{}' in step '{}'. Wrote updated model to '{}.inp'".
format(
    BC_NAME, SET_NAME, STEP_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

None.

11. Displacement / Rotation Boundary Condition

Category: Boundary Conditions

Description

Opens an existing model and applies a prescribed displacement and/or rotation to a named region, leaving unspecified degrees of freedom unconstrained.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SET_NAME must already exist as a defined set at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): Step this BC is created in
- **SET_NAME** (str): The exact, pre-existing assembly-level set name
- **BC_NAME** (str): Name for the new boundary condition
- **U1, U2, U3** (float or UNSET): Prescribed translations; UNSET leaves that DOF unconstrained
- **UR1, UR2, UR3** (float or UNSET): Prescribed rotations; UNSET leaves that DOF unconstrained
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, model.DisplacementBC(), job.writeInput()

Script

```
from abaqus import mdb
import os
from abaqusConstants import UNSET

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1"
SET_NAME = "Loaded-Edge"
BC_NAME = "Displacement-1"
U1 = 0.0
U2 = -2.5  # mm, prescribed displacement
U3 = UNSET
UR1 = UNSET
UR2 = UNSET
UR3 = UNSET
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

if SET_NAME not in assembly.sets.keys():
    raise ValueError("Set '{}' does not exist in the assembly.".format(SET_NAME))

region = assembly.sets[SET_NAME]
model.DisplacementBC(
    name=BC_NAME,
    createStepName=STEP_NAME,
    region=region,
    u1=U1, u2=U2, u3=U3,
    ur1=UR1, ur2=UR2, ur3=UR3,
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
```

```
export_job.writeInput(consistencyChecking=False)

print("Applied displacement BC '{}' to set '{}' in step '{}'. Wrote updated model to '{}.in
p'".format(
    BC_NAME, SET_NAME, STEP_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

For a shell/2D model, ur1/ur2/ur3 may not all be meaningful DOFs depending on element type -- confirm the model's element formulation before assuming all six DOFs apply.

12. Symmetry Boundary Condition

Category: Boundary Conditions

Description

Opens an existing model and applies a symmetry boundary condition (XSYMM, YSYMM, or ZSYMM) to a named region -- used when modeling only a symmetric half/quarter of a physical part.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SET_NAME must already exist as a defined set at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): Step this BC is created in
- **SET_NAME** (str): The exact, pre-existing assembly-level set name for the symmetry plane
- **BC_NAME** (str): Name for the new boundary condition
- **SYMMETRY_TYPE** (str): One of "XSYMM", "YSYMM", "ZSYMM"
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, model.XsymmBC()/YsymmBC()/ZsymmBC(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Initial"
SET_NAME = "Symmetry-Plane"
BC_NAME = "YSYMM-1"
SYMMETRY_TYPE = "YSYMM"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================
```

```
# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

if SET_NAME not in assembly.sets.keys():
    raise ValueError("Set '{}' does not exist in the assembly.".format(SET_NAME))

region = assembly.sets[SET_NAME]

if SYMMETRY_TYPE == "XSYMM":
    model.XsymmBC(name=BC_NAME, createStepName=STEP_NAME, region=region)
elif SYMMETRY_TYPE == "YSYMM":
    model.YsymmBC(name=BC_NAME, createStepName=STEP_NAME, region=region)
elif SYMMETRY_TYPE == "ZSYMM":
    model.ZsymmBC(name=BC_NAME, createStepName=STEP_NAME, region=region)
else:
    raise ValueError("SYMMETRY_TYPE must be one of XSYMM, YSYMM, ZSYMM")

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Applied {} symmetry BC '{}' to set '{}'. Wrote updated model to '{}.inp'".format(
    SYMMETRY_TYPE, BC_NAME, SET_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

None.

13. Concentrated Force on a Node Set

Category: Loads

Description

Opens an existing model and applies a concentrated force load to a pre-defined node set within a specific analysis step. Use this when a load acts at discrete node(s) rather than distributed over a surface.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SET_NAME must already exist as a defined set at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): The analysis step this load applies in
- **SET_NAME** (str): The exact, pre-existing assembly-level node set name
- **FORCE_COMPONENTS** (tuple of 3 floats): (Cf1, Cf2, Cf3) force components in N
- **LOAD_NAME** (str): Name for the new load
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, model.ConcentratedForce(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1"
SET_NAME = "Load-Region"
FORCE_COMPONENTS = (0.0, -500.0, 0.0)
LOAD_NAME = "AppliedForce-1"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
```

```
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

# This script assumes SET_NAME already exists as a defined set at the
# assembly level. It does not invent or guess this name -- confirm it
# exists before running this script.
if SET_NAME not in assembly.sets.keys():
    raise ValueError("Set '{}' does not exist in the assembly.".format(SET_NAME))

region = assembly.sets[SET_NAME]

model.ConcentratedForce(
    name=LOAD_NAME,
    createStepName=STEP_NAME,
    region=region,
    cf1=FORCE_COMPONENTS[0],
    cf2=FORCE_COMPONENTS[1],
    cf3=FORCE_COMPONENTS[2],
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Applied force '{}' to set '{}' in step '{}': ({}, {}, {}) N. Wrote updated model to
'{}.inp'".format(
    LOAD_NAME, SET_NAME, STEP_NAME, FORCE_COMPONENTS[0], FORCE_COMPONENTS[1], FORCE_COMPONE
NTS[2], OUTPUT_JOB_NAME
))
```

Notes / Caveats

If the set is defined at the PART level rather than the assembly level, reference it via the relevant instance instead of assembly.sets directly.

14. Pressure Load on a Surface

Category: Loads

Description

Opens an existing model and applies a uniform pressure load to a named surface.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SURFACE_NAME must already exist as a defined surface at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp

- **STEP_NAME** (str): The analysis step this load applies in
- **SURFACE_NAME** (str): The exact, pre-existing assembly-level surface name
- **LOAD_NAME** (str): Name for the new load
- **MAGNITUDE** (float): Pressure magnitude, in consistent units (e.g. MPa)
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.surfaces, model.Pressure(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1"
SURFACE_NAME = "Top-Surface"
LOAD_NAME = "Pressure-1"
MAGNITUDE = 2.5  # MPa
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

if SURFACE_NAME not in assembly.surfaces.keys():
    raise ValueError("Surface '{}' does not exist in the assembly.".format(SURFACE_NAME))

region = assembly.surfaces[SURFACE_NAME]
model.Pressure(name=LOAD_NAME, createStepName=STEP_NAME, region=region, magnitude=MAGNITUDE
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)
```

```
print("Applied pressure load '{}' of magnitude {} to surface '{}'. Wrote updated model to '
{}.inp'".format(
    LOAD_NAME, MAGNITUDE, SURFACE_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

A positive magnitude acts INTO the surface (compressive convention) -- confirm sign convention matches the intended physical direction before finalizing.

15. Distributed Line Load on Beam Elements

Category: Loads

Description

Opens an existing model and applies a distributed force per unit length to a set of beam elements/edges.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. SET_NAME must already exist as a defined set referencing beam elements or edges at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): The analysis step this load applies in
- **SET_NAME** (str): The exact, pre-existing set name for the target beam region
- **LOAD_NAME** (str): Name for the new load
- **MAGNITUDE** (float): Load magnitude per unit length (e.g. N/mm)
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, model.LineLoad(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1"
SET_NAME = "Beam-Elements"
LOAD_NAME = "LineLoad-1"
MAGNITUDE = 10.0  # N/mm
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
```

```
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

if SET_NAME not in assembly.sets.keys():
    raise ValueError("Set '{}' does not exist in the assembly.".format(SET_NAME))

region = assembly.sets[SET_NAME]
model.LineLoad(name=LOAD_NAME, createStepName=STEP_NAME, region=region, comp2=-MAGNITUDE)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Applied distributed line load '{}' of magnitude {} to '{}'. Wrote updated model to '
{}.inp'".format(
    LOAD_NAME, MAGNITUDE, SET_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

comp2 here applies the load along the global/local Y-axis as an example -- adjust comp1/comp2/comp3 to match the actual intended load direction for the specific task.

16. Gravity / Body Force Load

Category: Loads

Description

Opens an existing model and applies a gravity (body force) load across the entire model in a specified direction.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. Gravity applies globally, not to a named region, so no set/surface check is needed here.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): The analysis step this load applies in
- **LOAD_NAME** (str): Name for the new load

- **GRAVITY_MAGNITUDE** (float): Gravitational acceleration magnitude, in consistent units
- **DIRECTION** (tuple of 3 floats): Unit direction vector, e.g. (0, -1, 0) for -Y
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), model.Gravity(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Step-1"
LOAD_NAME = "Gravity-1"
GRAVITY_MAGNITUDE = 9810.0  # mm/s^2 -- consistent unit system: mm, tonne, N, MPa, s
DIRECTION = (0.0, -1.0, 0.0)
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]

model.Gravity(
    name=LOAD_NAME,
    createStepName=STEP_NAME,
    comp1=DIRECTION[0] * GRAVITY_MAGNITUDE,
    comp2=DIRECTION[1] * GRAVITY_MAGNITUDE,
    comp3=DIRECTION[2] * GRAVITY_MAGNITUDE,
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Applied gravity load '{}' of magnitude {} in direction {}. Wrote updated model to '{
}.inp'".format(
```

```
    LOAD_NAME, GRAVITY_MAGNITUDE, DIRECTION, OUTPUT_JOB_NAME
))
```

Notes / Caveats

Units matter critically here -- 9810 mm/s^2 is only correct if the rest of the model consistently uses mm/tonne/N/MPa/s. Confirm the model's unit system before using this value directly.

17. Tie Constraint Between Two Surfaces

Category: Interactions & Connections

Description

Opens an existing model and creates a tie constraint bonding a slave surface to a master surface, fully coupling their displacement.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. Both MASTER_SURFACE_NAME and SLAVE_SURFACE_NAME must already exist as defined surfaces at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **MASTER_SURFACE_NAME** (str): The exact, pre-existing master surface name
- **SLAVE_SURFACE_NAME** (str): The exact, pre-existing slave surface name
- **TIE_NAME** (str): Name for the new tie constraint
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.surfaces, model.Tie(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
MASTER_SURFACE_NAME = "Master-Surf"
SLAVE_SURFACE_NAME = "Slave-Surf"
TIE_NAME = "Tie-1"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )
```

```
mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

for surf_name in (MASTER_SURFACE_NAME, SLAVE_SURFACE_NAME):
    if surf_name not in assembly.surfaces.keys():
        raise ValueError("Surface '{}' does not exist in the assembly.".format(surf_name))

master = assembly.surfaces[MASTER_SURFACE_NAME]
slave = assembly.surfaces[SLAVE_SURFACE_NAME]

model.Tie(name=TIE_NAME, master=master, slave=slave, adjust=True)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Created tie constraint '{}' between '{}' (master) and '{}' (slave). Wrote updated mo
del to '{}.inp'".format(
    TIE_NAME, MASTER_SURFACE_NAME, SLAVE_SURFACE_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

adjust=True allows Abaqus to slightly adjust the slave surface nodes onto the master surface -- set this to False if the surfaces are already exactly coincident.

18. Surface-to-Surface Contact with Friction

Category: Interactions & Connections

Description

Opens an existing model, defines a frictional surface-to-surface contact interaction property, creates the contact interaction between two named surfaces, and persists the result.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. Both surfaces must already exist as defined surfaces at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **STEP_NAME** (str): Step the interaction is created in (often "Initial")
- **MASTER_SURFACE_NAME** (str): The exact, pre-existing master surface name
- **SLAVE_SURFACE_NAME** (str): The exact, pre-existing slave surface name
- **FRICTION_COEFF** (float): Coefficient of friction
- **INTERACTION_NAME** (str): Name for the new interaction
- **PROPERTY_NAME** (str): Name for the new contact property
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), model.ContactProperty(), property.TangentialBehavior(), property.NormalBehavior(), model.SurfaceToSurfaceContactStd(), job.writeInput()

Script

```
from abaqus import mdb
import os
import abaqusConstants as C

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
STEP_NAME = "Initial"
MASTER_SURFACE_NAME = "Contact-Master"
SLAVE_SURFACE_NAME = "Contact-Slave"
FRICTION_COEFF = 0.2
INTERACTION_NAME = "Contact-1"
PROPERTY_NAME = "ContactProp-1"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

for surf_name in (MASTER_SURFACE_NAME, SLAVE_SURFACE_NAME):
    if surf_name not in assembly.surfaces.keys():
        raise ValueError("Surface '{}' does not exist in the assembly.".format(surf_name))

interaction_property = model.ContactProperty(PROPERTY_NAME)
interaction_property.TangentialBehavior(formulation=C.PENALTY, table=((FRICTION_COEFF,),))
interaction_property.NormalBehavior(pressureOverclosure=C.HARD, allowSeparation=C.ON)

master = assembly.surfaces[MASTER_SURFACE_NAME]
slave = assembly.surfaces[SLAVE_SURFACE_NAME]

model.SurfaceToSurfaceContactStd(
    name=INTERACTION_NAME,
```

```
    createStepName=STEP_NAME,
    master=master,
    slave=slave,
    sliding=C.FINITE,
    interactionProperty=PROPERTY_NAME,
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Created surface-to-surface contact '{}' with friction coefficient {}. Wrote updated
model to '{}.inp'".format(
    INTERACTION_NAME, FRICTION_COEFF, OUTPUT_JOB_NAME
))
```

Notes / Caveats

sliding=FINITE allows large relative sliding between surfaces -- use SMALL instead if the surfaces are expected to stay in nearly the same relative position throughout the analysis, which converges more easily.

19. Kinematic Coupling Constraint

Category: Interactions & Connections

Description

Opens an existing model and rigidly couples a surface/region's motion to a single reference point, constraining all specified degrees of freedom identically -- commonly used to apply a load/BC through a single control point representing a rigid connector or bolt.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file. Both REFERENCE_POINT_SET_NAME and COUPLED_SET_NAME must already exist as defined sets at the assembly level.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to open
- **WORKING_DIRECTORY** (str): The task's working directory -- all output must land here, and nowhere else
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp
- **REFERENCE_POINT_SET_NAME** (str): The exact, pre-existing set name for the reference point
- **COUPLED_SET_NAME** (str): The exact, pre-existing set name for the coupled region
- **COUPLING_NAME** (str): Name for the new coupling constraint
- **OUTPUT_JOB_NAME** (str): Name used to write the updated model as OUTPUT_JOB_NAME.inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, model.Coupling(), job.writeInput()

Script

```
from abaqus import mdb
import os
from abaqusConstants import KINEMATIC, ON

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
```

```
WORKING_DIRECTORY = "/path/to/working_directory"
MODEL_NAME = "ImportedModel"
REFERENCE_POINT_SET_NAME = "RP-Set"
COUPLED_SET_NAME = "Coupled-Region"
COUPLING_NAME = "Coupling-1"
OUTPUT_JOB_NAME = "UpdatedModel"
# ====================

# All implicit-path Abaqus writes (job.writeInput()) use the process's
# current working directory -- there is no path argument for this call.
# Change into WORKING_DIRECTORY before any such call so output lands where
# it must, per the standing rule that every output goes inside
# WORKING_DIRECTORY and nowhere else.
os.chdir(WORKING_DIRECTORY)

# Guard against silently overwriting the very file this script just opened.
if OUTPUT_JOB_NAME + ".inp" == os.path.basename(INP_FILE_PATH):
    raise ValueError(
        "OUTPUT_JOB_NAME ('{}.inp') matches the input file's name -- choose a "
        "distinct output name to avoid overwriting the original file.".format(OUTPUT_JOB_NA
ME)
    )

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

for set_name in (REFERENCE_POINT_SET_NAME, COUPLED_SET_NAME):
    if set_name not in assembly.sets.keys():
        raise ValueError("Set '{}' does not exist in the assembly.".format(set_name))

control_point = assembly.sets[REFERENCE_POINT_SET_NAME]
surface_region = assembly.sets[COUPLED_SET_NAME]

model.Coupling(
    name=COUPLING_NAME,
    controlPoint=control_point,
    surface=surface_region,
    couplingType=KINEMATIC,
    u1=ON, u2=ON, u3=ON,
    ur1=ON, ur2=ON, ur3=ON,
)

export_job = mdb.Job(name=OUTPUT_JOB_NAME, model=MODEL_NAME)
export_job.writeInput(consistencyChecking=False)

print("Created kinematic coupling '{}' between '{}' and '{}'. Wrote updated model to '{}.in
p'".format(
    COUPLING_NAME, REFERENCE_POINT_SET_NAME, COUPLED_SET_NAME, OUTPUT_JOB_NAME
))
```

Notes / Caveats

REFERENCE_POINT_SET_NAME must reference an actual Reference Point feature, not an ordinary node -- confirm one exists (or create it) before using this pattern.

20. Submit a Job Directly from an Input File

Category: Job Submission

Description

Submits a complete, already-set-up .inp file directly as an analysis job, waits for completion, and checks the final status. This is simpler than reconstructing a Model object first -- job submission never needed one in the first place, only a complete .inp.

Prerequisites / Assumptions

INP_FILE_PATH must point to a COMPLETE, fully set-up .inp file (materials, sections, steps, loads, BCs, and mesh all already in place).

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the complete .inp file to submit
- **WORKING_DIRECTORY** (str): The task's working directory -- all job output must land here
- **JOB_NAME** (str): Name for the new job
- **NUM_CPUS** (int): Number of CPUs/domains to use

Key API Objects / Methods Used

mdb.JobFromInputFile(), job.submit(), job.waitForCompletion(), job.status

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
WORKING_DIRECTORY = "/path/to/working_directory"
JOB_NAME = "Job-1"
NUM_CPUS = 1
# ====================

# job.submit() uses the process's current working directory for its output
# files (.odb, .dat, .msg, .sta) -- there is no path argument for this call.
# Change into WORKING_DIRECTORY first so every output lands where it must.
os.chdir(WORKING_DIRECTORY)

job = mdb.JobFromInputFile(
    name=JOB_NAME,
    inputFileName=INP_FILE_PATH,
    numCpus=NUM_CPUS,
    numDomains=NUM_CPUS,
)

job.submit(consistencyChecking=False)
job.waitForCompletion()

status = job.status
print("Job '{}' finished with status: {}".format(JOB_NAME, status))

if status != "COMPLETED":
```

```
    raise RuntimeError("Job '{}' did not complete successfully -- status: {}".format(JOB_NA
ME, status))
```

Notes / Caveats

waitForCompletion() blocks until the job finishes -- for a long-running analysis, ensure the calling process's own timeout is set accordingly. The resulting .odb will be named JOB_NAME.odb in the working directory.

21. Peak von Mises Stress Extraction

Category: Post-Processing

Description

Opens a completed .odb and reports the peak von Mises stress value and the element it occurs at, for a specific step and frame.

Prerequisites / Assumptions

ODB_PATH must point to a completed, readable .odb file containing stress ('S') field output for STEP_NAME.

Inputs Required

- **ODB_PATH** (str): Absolute path to the .odb file
- **STEP_NAME** (str): The analysis step to extract from
- **FRAME_INDEX** (int): Frame index within the step; -1 for the last frame

Key API Objects / Methods Used

odbAccess.openOdb(), field.getScalarField(invariant=MISES)

Script

```
from odbAccess import openOdb
from abaqusConstants import MISES

# === USER INPUTS ===
ODB_PATH = "/path/to/results.odb"
STEP_NAME = "Step-1"
FRAME_INDEX = -1
# ====================

odb = openOdb(path=ODB_PATH, readOnly=True)

try:
    step = odb.steps[STEP_NAME]
    frame = step.frames[FRAME_INDEX]

    stress_field = frame.fieldOutputs["S"]
    mises_field = stress_field.getScalarField(invariant=MISES)

    peak_stress = 0.0
    peak_location = None
    for value in mises_field.values:
        if value.data > peak_stress:
            peak_stress = value.data
            peak_location = value.elementLabel

    print("Peak von Mises stress in step '{}': {} at element {}".format(
        STEP_NAME, peak_stress, peak_location
    ))
finally:
    odb.close()
```

Notes / Caveats

Always close the odb in a finally block -- an odb left open can lock the file for other processes (including a later re-run of the same analysis).

22. Reaction Force Extraction at a Node Set

Category: Post-Processing

Description

Opens a completed .odb and extracts/sums the reaction force components at a specific named node set for a given step and frame.

Prerequisites / Assumptions

ODB_PATH must point to a completed .odb. SET_NAME must already exist as a node set in the odb's assembly (it will if it was defined in the .inp before the job ran).

Inputs Required

- **ODB_PATH** (str): Absolute path to the .odb file
- **STEP_NAME** (str): The analysis step to extract from
- **SET_NAME** (str): The exact, pre-existing node set name
- **FRAME_INDEX** (int): Frame index within the step; -1 for the last frame

Key API Objects / Methods Used

odbAccess.openOdb(), odb.rootAssembly.nodeSets, field.getSubset()

Script

```
from odbAccess import openOdb

# === USER INPUTS ===
ODB_PATH = "/path/to/results.odb"
STEP_NAME = "Step-1"
SET_NAME = "Fixed-Support"
FRAME_INDEX = -1
# ====================

odb = openOdb(path=ODB_PATH, readOnly=True)

try:
    assembly = odb.rootAssembly

    if SET_NAME not in assembly.nodeSets.keys():
        raise ValueError("Node set '{}' does not exist in the odb assembly.".format(SET_NAM
E))

    node_set = assembly.nodeSets[SET_NAME]

    step = odb.steps[STEP_NAME]
    frame = step.frames[FRAME_INDEX]

    rf_field = frame.fieldOutputs["RF"]
    rf_subset = rf_field.getSubset(region=node_set)

    total_rf = [0.0, 0.0, 0.0]
```

```
    for value in rf_subset.values:
        for i in range(3):
            total_rf[i] += value.data[i]

    print("Total reaction force at set '{}' in step '{}': ({}, {}, {}) N".format(
        SET_NAME, STEP_NAME, total_rf[0], total_rf[1], total_rf[2]
    ))
finally:
    odb.close()
```

Notes / Caveats

If SET_NAME was defined at the PART level rather than the assembly level in the original .inp, check odb.rootAssembly.instances[].nodeSets instead.

23. Nodal Displacement Time-History

Category: Post-Processing

Description

Opens a completed .odb and extracts the displacement of a monitored node across every frame of a step, producing a time-history.

Prerequisites / Assumptions

ODB_PATH must point to a completed .odb. SET_NAME must already exist as a node set containing exactly the node(s) to monitor.

Inputs Required

- **ODB_PATH** (str): Absolute path to the .odb file
- **STEP_NAME** (str): The analysis step to extract from
- **SET_NAME** (str): The exact, pre-existing node set name for the monitored node

Key API Objects / Methods Used

odbAccess.openOdb(), odb.rootAssembly.nodeSets, step.frames, field.getSubset()

Script

```
from odbAccess import openOdb

# === USER INPUTS ===
ODB_PATH = "/path/to/results.odb"
STEP_NAME = "Step-1"
SET_NAME = "Monitor-Node"
# ====================

odb = openOdb(path=ODB_PATH, readOnly=True)

try:
    assembly = odb.rootAssembly

    if SET_NAME not in assembly.nodeSets.keys():
        raise ValueError("Node set '{}' does not exist in the odb assembly.".format(SET_NAM
E))
```

```
    node_set = assembly.nodeSets[SET_NAME]
    step = odb.steps[STEP_NAME]

    time_history = []
    for frame in step.frames:
        disp_field = frame.fieldOutputs["U"]
        disp_subset = disp_field.getSubset(region=node_set)
        if disp_subset.values:
            value = disp_subset.values[0]
            time_history.append((frame.frameValue, value.data))

    print("Displacement time-history for set '{}':".format(SET_NAME))
    for time_val, displacement in time_history:
        print("  t={}: U=({}, {}, {})".format(time_val, displacement[0], displacement[1], d
isplacement[2]))
finally:
    odb.close()
```

Notes / Caveats

If SET_NAME contains more than one node, only the first value in the subset is reported here -- extend the loop over disp_subset.values if multiple nodes need to be tracked simultaneously.

24. Write Extracted Stress Results to CSV

Category: Post-Processing

Description

Opens a completed .odb, extracts von Mises stress values for every integration point in a frame, and writes them to a CSV file -- useful when the user wants a downloadable results table rather than an in-chat summary.

Prerequisites / Assumptions

ODB_PATH must point to a valid, completed .odb file. OUTPUT_CSV_PATH's parent directory must exist and be writable -- this should be the task's WORKING_DIRECTORY.

Inputs Required

- **ODB_PATH** (str): Absolute path to the .odb file
- **STEP_NAME** (str): The analysis step to extract from
- **FRAME_INDEX** (int): Frame index within the step; -1 for the last frame
- **OUTPUT_CSV_PATH** (str): Absolute path for the output CSV file -- must be inside WORKING_DIRECTORY

Key API Objects / Methods Used

odbAccess.openOdb(), field.getScalarField(), csv.writer()

Script

```
import csv
from odbAccess import openOdb
from abaqusConstants import MISES

# === USER INPUTS ===
ODB_PATH = "/path/to/results.odb"
STEP_NAME = "Step-1"
FRAME_INDEX = -1
```

```
OUTPUT_CSV_PATH = "/path/to/working_directory/stress_results.csv"  # must be inside WORKING
_DIRECTORY
# ====================

odb = openOdb(path=ODB_PATH, readOnly=True)

try:
    step = odb.steps[STEP_NAME]
    frame = step.frames[FRAME_INDEX]

    stress_field = frame.fieldOutputs["S"]
    mises_field = stress_field.getScalarField(invariant=MISES)

    with open(OUTPUT_CSV_PATH, "w") as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(["ElementLabel", "IntegrationPoint", "MisesStress"])
        for value in mises_field.values:
            writer.writerow([value.elementLabel, value.integrationPoint, value.data])

    print("Wrote von Mises stress results to '{}'".format(OUTPUT_CSV_PATH))
finally:
    odb.close()
```

Notes / Caveats

OUTPUT_CSV_PATH must be built by joining WORKING_DIRECTORY with a filename -- never a hardcoded or relative path elsewhere.

25. List All Defined Sets and Surfaces in an Existing Model

Category: File Utility

Description

Opens an existing .inp file and prints every set and surface currently defined at the assembly level, and every set defined at the part level for each part -- a direct way to confirm whether a specific named region actually exists in the model before targeting it in another script.

Prerequisites / Assumptions

INP_FILE_PATH must point to a valid, existing .inp file.

Inputs Required

- **INP_FILE_PATH** (str): Absolute path to the existing .inp file to inspect
- **MODEL_NAME** (str): Name to assign the model reconstructed from the .inp

Key API Objects / Methods Used

mdb.ModelFromInputFile(), assembly.sets, assembly.surfaces, model.parts, part.sets

Script

```
from abaqus import mdb

# === USER INPUTS ===
INP_FILE_PATH = "/path/to/working_directory/current_model.inp"
MODEL_NAME = "InspectedModel"
# ====================

mdb.ModelFromInputFile(name=MODEL_NAME, inputFileName=INP_FILE_PATH)
model = mdb.models[MODEL_NAME]
assembly = model.rootAssembly

print("=== Assembly-level Sets ===")
for set_name in assembly.sets.keys():
    print("  - {}".format(set_name))

print("=== Assembly-level Surfaces ===")
for surf_name in assembly.surfaces.keys():
    print("  - {}".format(surf_name))

print("=== Part-level Sets (by part) ===")
for part_name, part in model.parts.items():
    print("  Part: {}".format(part_name))
    for set_name in part.sets.keys():
        print("    - {}".format(set_name))
```

Notes / Caveats

Run this FIRST whenever a task references a specific set/surface by description rather than by an exact, already-confirmed name -- this is the direct, practical way to check before targeting a region with any other script in this document.

26. Write an Updated Input File (.inp) from the Current In-Memory Model

Category: File Utility

Description

Writes out the current state of an IN-MEMORY model (already open in this same script's session -- e.g. right after a geometry-import-and-mesh sequence) as a fresh .inp file. Use this as the final step of a combined script that performs several operations together, rather than as a standalone script on its own.

Prerequisites / Assumptions

MODEL_NAME must already exist and be fully set up as intended, IN THIS SAME SCRIPT'S session -- this script does not open anything from a file itself.

Inputs Required

- **MODEL_NAME** (str): Name of the model database entry to export (already in memory)
- **JOB_NAME** (str): Name used for the job object created solely to generate the .inp
- **OUTPUT_DIRECTORY** (str): Directory the .inp should end up in -- must be WORKING_DIRECTORY

Key API Objects / Methods Used

mdb.Job(), job.writeInput()

Script

```
from abaqus import mdb
import os

# === USER INPUTS ===
MODEL_NAME = "Model-1"
JOB_NAME = "ModelExport-1"
OUTPUT_DIRECTORY = "/path/to/working_directory"
# ====================

# writeInput() uses the process's current working directory -- there is no
# path argument for this call. Change into OUTPUT_DIRECTORY first so the
# .inp actually lands where it must, rather than wherever the process
# happened to start.
os.chdir(OUTPUT_DIRECTORY)

job = mdb.Job(name=JOB_NAME, model=MODEL_NAME)
job.writeInput(consistencyChecking=False)

expected_path = os.path.join(OUTPUT_DIRECTORY, JOB_NAME + ".inp")
print("Wrote updated input file to '{}'".format(expected_path))
```

Notes / Caveats

Every one of the modification scripts elsewhere in this document (materials, steps, BCs, loads, interactions) already includes this exact chdir-and-write-out pattern internally at its own end -- use THIS standalone version only when combining several building-block operations into ONE script and exporting once at the true end, not after every individual operation.